Tab 1

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Nexus Cloud IDE

Version: 1.0

Document Status: Approved for Development

Prepared By: Software Architecture Team

Project Type: Premium Full Stack Cloud Development Platform (SaaS)

---

1. Executive Summary

1.1 Introduction

Nexus Cloud IDE is a next-generation cloud development platform designed to deliver a complete software development ecosystem through a modern web browser. The platform combines source code editing, artificial intelligence, project management, version control, collaboration, deployment, database management, and development utilities into a single unified workspace.

Unlike traditional online code editors that focus primarily on writing and executing code, Nexus Cloud IDE aims to provide an end-to-end development experience capable of supporting individuals, educational institutions, startups, professional development teams, and enterprise organizations.

The primary objective of the platform is to eliminate the need for developers to continuously switch between multiple applications such as code editors, terminals, deployment dashboards, database tools, Git clients, AI assistants, and collaboration software. Every essential workflow should be accessible from within one consistent environment.

The platform is designed with a strong emphasis on usability, scalability, security, maintainability, and premium user experience. Every interaction within the application should feel responsive, intuitive, and professional.

1.2 Vision Statement

To become a unified cloud development platform that enables developers to design, build, collaborate, test, deploy, and manage software projects from a single intelligent workspace.

1.3 Mission Statement

To simplify software development by integrating every essential development workflow into one seamless browser-based platform while maintaining production-level performance, scalability, and developer experience.

1.4 Product Philosophy

One Platform. Every Developer. Every Workflow.

Nexus Cloud IDE is built around the philosophy that software development should not require constant context switching between multiple applications.

Instead of opening separate tools for editing code, version control, deployment, artificial intelligence, databases, collaboration, terminals, and project management, developers should be able to perform every essential task inside one cohesive platform.

1.5 Problem Statement

Modern software development often requires developers to use multiple disconnected tools throughout a single development cycle.

Typical workflows involve switching between:

- Code Editor
- Terminal
- Git Client
- GitHub
- Database Tools
- Browser Preview
- Deployment Dashboard
- Documentation

- Artificial Intelligence Tools
- Team Communication Platforms

Frequent context switching increases cognitive load, reduces productivity, interrupts focus, and creates unnecessary complexity.

Students face an even greater challenge because learning numerous independent tools simultaneously can become overwhelming.

Nexus Cloud IDE addresses these challenges by consolidating the entire development workflow into a single integrated platform.

1.6 Objectives

The primary objectives of Nexus Cloud IDE are:

- Provide a complete cloud-based software development environment.
- Deliver a premium SaaS experience comparable to leading cloud IDE platforms.
- Reduce development complexity by centralizing workflows.
- Simplify project management through integrated workspaces.
- Enable seamless collaboration between developers.
- Integrate AI throughout the development lifecycle.
- Support one-click deployment for supported platforms.
- Offer a scalable architecture capable of supporting future enterprise features.
- Maintain high performance and responsiveness across all modules.
- Provide a clean, intuitive, and visually consistent interface.

1.7 Product Category

Nexus Cloud IDE is classified as a:

Cloud Development Platform

It is not classified as:

- Online Compiler
- Simple Code Editor
- IDE Clone

Instead, it represents a complete development ecosystem designed for modern software engineering.

1.8 Target Audience

Nexus Cloud IDE is intended for:

Students

Learners who require a simplified development environment without installing numerous software applications.

Beginner Developers

Users learning web development, programming languages, version control, and deployment workflows.

Professional Developers

Software engineers seeking an integrated browser-based development environment.

Freelancers

Independent developers managing multiple client projects from one workspace.

Startup Teams

Small engineering teams collaborating on shared projects.

Enterprise Organizations

Organizations requiring centralized project management, collaboration, deployment, and administrative control.

1.9 Business Goals

The platform aims to:

- Increase developer productivity.
- Reduce workflow interruptions.
- Improve collaboration efficiency.
- Simplify software deployment.
- Deliver AI-assisted development.
- Provide an enterprise-ready architecture.

- Establish a premium developer experience.
- Demonstrate production-quality engineering standards.

1.10 Success Criteria

The project will be considered successful when it achieves the following:

- Developers can create and manage projects entirely within Nexus Cloud IDE.
- AI assists developers throughout the development process.
- Teams collaborate effectively through shared workspaces.
- Projects can be deployed with minimal manual configuration.
- The interface remains responsive and intuitive under normal workloads.
- The architecture supports future expansion without major redesign.
- The platform demonstrates production-level software engineering practices.

1.11 Scope

The initial version of Nexus Cloud IDE includes:

- User Authentication
- Organization Management
- Workspace Management
- Project Management
- File Explorer
- Monaco Code Editor
- Shared Terminal
- Artificial Intelligence Assistant
- Git Integration
- Database Explorer
- One-Click Deployment
- Collaboration Features
- Admin Dashboard
- Billing (Free and Pro)
- Settings
- Themes
- Project Templates

Future versions may introduce:

- AI Agents
- Multi-Workspace Organizations
- Team Permissions Expansion

- Advanced Monitoring
- Marketplace
- Third-Party Plugin Ecosystem
- Browser Extensions
- Mobile Companion Application

Tab 2

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Nexus Cloud IDE

Version: 1.0

Document Status: Approved for Development

---

2. Product Overview

2.1 Product Description

Nexus Cloud IDE is a browser-based cloud development platform that provides developers with a complete software development environment through a single unified interface.

The platform enables users to create, edit, execute, manage, collaborate, and deploy software projects without depending on multiple external development tools.

Rather than functioning as a simple online code editor, Nexus Cloud IDE integrates project management, artificial intelligence, version control, deployment, collaboration, database management, and development utilities into one premium SaaS platform.

The application is designed with scalability in mind, allowing it to evolve from supporting individual developers into serving startup teams, educational institutions, and enterprise organizations.

2.2 Product Positioning

Nexus Cloud IDE positions itself as a modern cloud development ecosystem rather than a standalone code editor.

The platform combines concepts inspired by professional development environments while establishing its own identity through a unified workflow, integrated AI assistance, and premium user experience.

The product focuses on reducing developer context switching by consolidating essential development workflows into a single browser-based platform.

2.3 Core Principles

Every decision made during development must follow these principles.

1. Unified Development Experience

Developers should rarely need to leave Nexus Cloud IDE during the software development lifecycle.

---

2. Premium User Experience

Every screen must appear polished, professional, and production-ready.

The interface should resemble a commercial SaaS application rather than an academic project.

---

3. Simplicity First

Powerful features should remain intuitive for beginners while still providing advanced capabilities for experienced developers.

---

4. Consistency

Navigation, layouts, colors, spacing, typography, interactions, and behavior must remain consistent throughout the platform.

---

5. Performance

Every interaction should feel immediate.

Animations, transitions, searches, navigation, and rendering must remain smooth.

---

6. Scalability

The architecture must support future modules without requiring major restructuring.

---

7. Security

Security must be considered during every phase of development rather than being added afterward.

2.4 Product Goals

The platform shall provide:

- Complete browser-based development
- AI-assisted programming
- Workspace management
- Team collaboration
- Git integration
- Database management
- Deployment tools
- Premium user interface
- Enterprise-ready architecture
- High scalability

2.5 Target Platforms

Nexus Cloud IDE shall support:

Desktop

Primary development platform.

Recommended minimum resolution:

1920 × 1080

Laptop

Fully responsive interface.

Tablet

Optimized layout.

Panels should intelligently collapse.

Mobile

Project viewing

Settings

Basic management

Notifications

Administrative tasks

Mobile editing is intentionally limited to preserve developer experience.

2.6 Stakeholders

The primary stakeholders include:

Platform Users

Developers using Nexus Cloud IDE for software development.

Organization Owners

Users responsible for managing organizations, billing, permissions, and workspaces.

Administrators

Platform administrators responsible for system management, monitoring, moderation, analytics, and user support.

Development Team

Engineers responsible for maintaining and expanding the platform.

2.7 User Roles

The platform shall support the following roles.

Owner

The highest authority within an organization.

Permissions include:

- Delete organization
- Manage billing
- Invite users
- Remove users
- Create workspaces
- Delete workspaces
- Manage permissions
- Configure organization settings

Administrator

Responsible for organizational management.

Permissions include:

- Invite members
- Manage projects
- Manage workspaces
- Configure settings
- View analytics

Cannot delete the organization.

Maintainer

Responsible for project maintenance.

Permissions include:

- Manage repositories
- Manage deployments
- Review collaboration requests
- Configure project settings

Developer

Standard contributor.

Permissions include:

- Edit code
- Create projects
- Use AI
- Deploy projects
- Access databases
- Use terminal
- Collaborate

Viewer

Read-only access.

Permissions include:

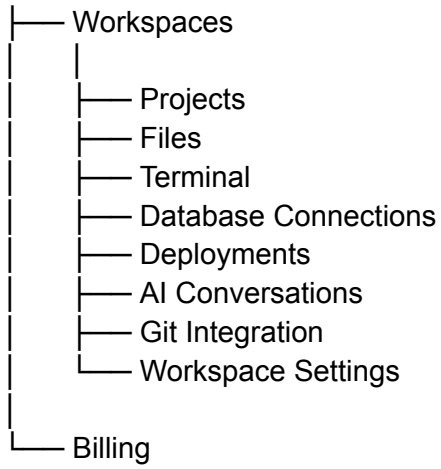
- View projects
- View files
- View documentation

Cannot edit project resources.

2.8 Organization Model

The platform shall organize resources using the following hierarchy.

Organization



This architecture allows the platform to scale naturally from individual developers to enterprise organizations.

2.9 Workspace

A Workspace represents a complete development environment.

A workspace contains:

- Projects
- Files
- AI conversations
- Shared terminal
- Database connections
- Deployment history
- Git configuration
- Workspace settings
- Collaborators

Projects are shared through the workspace rather than individually.

2.10 Project

A project represents an individual software application within a workspace.

Each project includes:

- Source code
- Assets

- Configuration
- Dependencies
- Git repository
- Build configuration
- Deployment configuration
- Environment variables
- Runtime settings

2.11 Product Philosophy

The success of Nexus Cloud IDE will not be measured solely by the number of features it provides.

Instead, success will be determined by the quality of the developer experience.

When choosing between introducing additional functionality or improving usability, usability and consistency shall always take priority.

Every interaction should communicate quality, reliability, and professionalism.

2.12 Product Identity

Nexus Cloud IDE is designed to become:

- A professional cloud development platform.
- A complete development ecosystem.
- A portfolio-grade production application.
- A scalable SaaS product.
- A foundation capable of supporting future enterprise features.

The platform shall never be positioned as merely an online compiler or a browser-based code editor.

Tab 3

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Nexus Cloud IDE

Version: 1.0

Document Status: Approved for Development

---

3. Functional Requirements

3.1 Introduction

This section defines the functional capabilities of Nexus Cloud IDE.

Each module described below represents a core feature of the platform and specifies the expected behavior, responsibilities, permissions, and interactions.

All functional requirements described in this section are mandatory unless explicitly marked as a future enhancement.

Module 1 — Authentication

Purpose

Provide secure authentication and account management for all users accessing Nexus Cloud IDE.

Features

User Registration

The platform shall allow users to create an account using:

- Email & Password
- Google Authentication
- GitHub Authentication

Email Verification

Every newly registered account must verify its email address before accessing the platform.

Unverified accounts shall not be permitted to create organizations or workspaces.

User Login

The platform shall allow users to authenticate using:

- Email & Password
- Google
- GitHub

Password Security

Passwords shall:

- Never be stored in plain text.
- Be securely hashed before storage.
- Meet configurable minimum complexity requirements.

Forgot Password

Users shall be able to request password reset links through email.

User Profile

Each user profile shall contain:

- Full Name
- Username
- Email Address
- Profile Picture

- Account Creation Date
- Connected Providers
- Account Settings

Logout

Users shall be able to securely terminate all active sessions.

Future Features

- Two-Factor Authentication
- Magic Link Login
- Microsoft Login
- Passkeys

Module 2 — Organization Management

Purpose

Organizations provide the highest level of resource ownership.

Every workspace belongs to an organization.

Functional Requirements

Users shall be able to:

- Create Organizations
- Rename Organizations
- Delete Organizations
- Upload Organization Logo
- Invite Members
- Remove Members
- Assign Roles
- View Organization Members
- Configure Organization Settings

Organization Roles

Supported roles:

- Owner
- Administrator
- Maintainer
- Developer
- Viewer

Each role shall enforce permission-based access throughout the platform.

Module 3 — Workspace Management

Purpose

A Workspace represents a complete development environment.

Functional Requirements

Users shall be able to:

- Create Workspace
- Rename Workspace
- Delete Workspace
- Share Workspace
- Archive Workspace
- Restore Workspace
- View Workspace Members
- Configure Workspace Settings

Each Workspace shall include:

- Projects
- Shared Terminal
- AI Conversations
- Git Configuration
- Database Connections
- Deployments
- Workspace Settings

- Collaborators

Module 4 — Project Management

Purpose

Projects represent software applications inside workspaces.

Functional Requirements

Users shall be able to:

- Create Project
- Rename Project
- Delete Project
- Duplicate Project
- Archive Project
- Restore Project
- Favorite Project
- Search Projects
- Clone Project
- Export Project
- Import Project

Supported Project Templates

Version 1 shall include:

- HTML
- CSS
- JavaScript
- React
- Next.js
- Node.js
- Express.js
- React + Express
- Python
- Java
- C++

- Empty Project

Future templates may be introduced without modifying existing projects.

Project Import

Supported methods:

- GitHub Repository
- Git URL
- Local Folder

Project Export

Supported methods:

- ZIP Download
- GitHub Push
- Docker Image
- Project Download

Module 5 — Dashboard

Purpose

The Dashboard serves as the primary landing page after authentication.

Dashboard Features

Display:

- Recent Projects
- Favorite Projects
- Organization List
- Workspace List
- Recent Activity
- Deployment Status

- AI Activity Summary
- Storage Usage
- Quick Actions

Quick Actions include:

- New Project
- New Workspace
- Import Repository
- Join Workspace

Module 6 — File Explorer

Purpose

Provide hierarchical navigation of project files.

Functional Requirements

Support:

- Nested Folders
- Create File
- Create Folder
- Rename
- Delete
- Duplicate
- Copy Path
- Collapse Folder
- Expand Folder
- File Icons
- Folder Icons

Right Click Menu

Every file shall support:

- Rename

- Delete
- Duplicate
- Copy Path

Every folder shall support:

- New File
- New Folder
- Rename
- Delete

Drag-and-drop support is not included in Version 1.

Module 7 — Monaco Code Editor

Purpose

Provide a professional browser-based code editing experience.

Features

The editor shall support:

- Monaco Editor
- Syntax Highlighting
- Multiple Tabs
- Auto Save
- Split Editor
- Line Numbers
- Bracket Matching
- Auto Indentation
- Auto Closing Tags
- Code Folding
- Breadcrumb Navigation
- Find & Replace
- Minimap
- Word Wrap
- Sticky Cursor
- UTF-8 Encoding
- LF Line Endings

- Unsaved Change Indicators

Editor Tabs

Users shall:

- Open multiple files.
- Close files.
- Reorder tabs.
- Switch between tabs efficiently.

Auto Save

Projects shall automatically save changes at regular intervals.

Manual saving shall remain available through keyboard shortcuts.

Split Editor

Users shall be able to view two files simultaneously.

Future versions may support multiple editor groups.

Future Enhancements

- Collaborative editing
- Inline Git Blame
- Code Lens
- Multi-cursor editing
- Advanced refactoring tools

Module 8 — Shared Terminal

Purpose

Provide an integrated terminal environment inside each workspace.

Functional Requirements

Each workspace shall contain:

- Shared Terminal
- Multiple Terminal Instances
- Terminal Tabs

Supported actions:

- Create Terminal
- Rename Terminal
- Close Terminal
- Restart Terminal
- Clear Terminal

Terminal sessions shall remain active while users navigate between platform pages.

Terminal output shall support:

- ANSI Colors
- Scroll History
- Copy Output

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Nexus Cloud IDE

Version: 1.0

Document Status: Approved for Development

---

3. Functional Requirements (Part 2)

Module 9 — AI Assistant

Purpose

The AI Assistant serves as an intelligent development companion integrated throughout the platform. It assists users in understanding, writing, improving, debugging, and documenting code while remaining aware of the active project context.

Unlike standalone AI chatbots, the AI Assistant shall operate as a workspace-aware system capable of understanding the project's structure and assisting users during every stage of software development.

Functional Requirements

The AI Assistant shall support:

- Project-aware conversations
- Current file understanding
- Selected code understanding
- Terminal output analysis
- Error explanation
- Documentation assistance
- Code generation suggestions
- Code optimization suggestions
- Unit test generation
- Database schema suggestions
- Deployment configuration suggestions
- README generation
- Git commit message generation

Workspace Memory

Each workspace shall maintain its own AI conversation history.

AI conversations shall remain isolated between workspaces.

AI Interaction Rules

The AI Assistant shall:

- Suggest changes only
- Never modify project files automatically
- Require user approval before applying generated code

AI Interface

The AI panel shall support:

- Chat history
- Markdown rendering
- Syntax highlighted code blocks
- Copy responses
- Regenerate responses
- Clear conversation
- Export conversation

Future Enhancements

- AI Agents
- Autonomous bug fixing
- Intelligent project planning
- Multi-agent collaboration
- Voice interaction

Module 10 — Git Integration

Purpose

Provide built-in version control functionality without requiring external Git clients.

Functional Requirements

Users shall be able to:

- Initialize Git repositories
- Authenticate with GitHub
- Commit changes
- Push commits
- Pull updates
- Create branches
- Merge branches
- View commit history

Git Interface

The Git panel shall display:

- Changed files
- Staged changes
- Commit message input
- Current branch
- Repository status

GitHub Integration

The platform shall support:

- GitHub Authentication
- Repository linking
- Repository creation
- Push to GitHub
- Pull from GitHub

Future Enhancements

- Pull Requests
- GitHub Issues
- Release Management
- Diff Viewer
- Cherry-pick
- Rebase

Module 11 — Database Explorer

Purpose

Provide a graphical interface for viewing connected databases and executing queries.

Supported Databases

Version 1 shall support:

- MongoDB
- PostgreSQL
- SQLite

Future versions may support:

- Supabase
- Firebase
- MySQL

Functional Requirements

Users shall be able to:

- Connect databases
- Browse collections
- Browse tables
- View documents
- View rows
- Execute MongoDB queries
- Execute SQL queries

Database editing is intentionally excluded from Version 1.

Module 12 — One-Click Deployment

Purpose

Enable developers to deploy projects directly from Nexus Cloud IDE with minimal configuration.

Supported Providers

Version 1:

- Vercel
- Docker
- GitHub Pages

Future Versions:

- Netlify
- Railway
- Render
- AWS
- Azure
- Google Cloud

Deployment Workflow

Users shall be able to:

- Configure deployment
- Start deployment
- View deployment status
- View deployment logs
- Copy deployment URL
- View deployment history

Deployment Results

After successful deployment, users shall receive:

- Live URL
- Build logs
- Deployment history

Module 13 — Collaboration

Purpose

Allow multiple users to work together inside shared workspaces.

Functional Requirements

The collaboration system shall support:

- Workspace invitations
- Role-based access
- Shared terminal
- Shared preview
- Comments
- Follow collaborator

Permission Requests

Users without edit permission shall request editing access from authorized members.

Future Enhancements

- Live cursors
- Simultaneous editing
- Voice collaboration
- Screen sharing
- Collaborative debugging

Module 14 — Admin Dashboard

Purpose

Provide centralized management capabilities for platform administrators.

Functional Requirements

Administrators shall manage:

- Users
- Organizations
- Projects
- Billing
- AI Usage
- Storage
- Reports
- Analytics
- System Health

Administrative Actions

Administrators shall be able to:

- Suspend users
- Delete accounts
- View storage consumption
- View AI usage statistics
- Manage subscriptions
- Generate reports

Module 15 — Billing & Subscription

Purpose

Manage platform subscriptions and feature access.

Subscription Plans

Free

Limitations:

- Limited number of projects
- Limited storage
- Limited collaboration

Pro

Provides:

- Increased project limits
- Increased storage
- Full collaboration features
- Premium workspace capabilities

Future plans may include Team and Enterprise subscriptions.

Module 16 — Settings

Functional Requirements

Users shall configure:

Account

- Profile
- Password
- Connected Accounts

Workspace

- Workspace Name

- Default Language
- Terminal Preferences

Editor

- Font Size
- Font Family
- Word Wrap
- Auto Save

Appearance

- Theme
- Accent Color (future)
- Editor Preferences

Module 17 — Themes

Version 1 shall include:

- Obsidian (Default)
- Forest

Future versions may introduce additional professionally designed themes.

Module 18 — Toast Notifications

The platform shall display toast notifications for important actions.

Examples include:

- Login Successful
- Project Saved
- Deployment Complete
- Workspace Created
- Project Deleted
- Git Push Successful

Toast notifications shall be temporary, unobtrusive, and dismiss automatically after a short duration.

Module 19 — Search & Quick Open

Version 1 shall support:

- File Search
- Quick Open (`Ctrl + P`)

Future versions may include:

- Global Search
- Command Palette (`Ctrl + Shift + P`)
- Symbol Search
- Workspace Search

Module 20 — Future Modules

The following modules are intentionally excluded from Version 1 but are part of the long-term product roadmap:

- AI Agents
- Third-Party Plugin Marketplace
- Multi-Workspace Organizations
- Team Permission Expansion
- Advanced Monitoring
- Real-Time Collaborative Editing
- Voice Collaboration
- Mobile Companion Application
- Browser Extension
- Cloud Marketplace

Tab 4

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Nexus Cloud IDE

Version: 1.0

Document Status: Approved for Development

---

4. Non-Functional Requirements

4.1 Introduction

Non-functional requirements define the quality attributes of Nexus Cloud IDE. These requirements specify how the platform should perform, scale, remain secure, and provide a consistent user experience under normal operating conditions.

All modules developed for Nexus Cloud IDE shall comply with the standards defined in this section.

4.2 Performance Requirements

The platform shall prioritize responsiveness and smooth interactions.

User Interface Performance

The application shall:

- Maintain smooth navigation between pages.
- Minimize unnecessary re-renders.
- Use lazy loading where appropriate.
- Load UI components efficiently.
- Display loading indicators during asynchronous operations.
- Avoid blocking the main UI thread.

Editor Performance

The code editor shall:

- Open files without noticeable delay.
- Handle large source files efficiently.
- Preserve smooth scrolling.
- Maintain responsive typing performance.
- Support syntax highlighting without affecting usability.

Workspace Performance

Workspace switching shall:

- Restore the previous session quickly.
- Load only required resources.
- Delay non-essential requests until needed.

Network Performance

The application shall:

- Cache reusable resources.
- Reduce unnecessary API requests.
- Optimize payload sizes.
- Compress responses where applicable.

4.3 Scalability Requirements

The architecture shall support future growth without major redesign.

The platform shall support:

- Thousands of registered users.
- Multiple organizations per user.
- Multiple workspaces per organization.
- Multiple projects per workspace.
- Expansion of future modules.
- Horizontal scaling of backend services.

The architecture shall remain modular to simplify future feature additions.

4.4 Availability

The platform should remain available whenever users require access.

Temporary service interruptions should:

- Display informative error messages.
- Avoid data corruption.
- Recover gracefully after service restoration.

4.5 Reliability

The platform shall maintain consistent behavior.

The system shall:

- Prevent unexpected crashes.
- Preserve project integrity.
- Handle unexpected failures gracefully.
- Log server-side failures.
- Validate critical operations before execution.

4.6 Maintainability

The codebase shall be designed for long-term maintenance.

Requirements include:

- Modular architecture.
- Feature-based organization.
- Reusable UI components.
- Consistent naming conventions.
- Minimal code duplication.
- Clear separation of concerns.
- Comprehensive documentation.

4.7 Usability

The platform shall remain approachable for beginners while providing advanced capabilities for experienced developers.

The interface shall:

- Minimize unnecessary complexity.
- Provide intuitive navigation.
- Use consistent terminology.
- Display meaningful error messages.
- Reduce repetitive actions.

4.8 Accessibility

Nexus Cloud IDE shall follow modern accessibility principles.

The interface shall support:

- Keyboard navigation.
- Visible focus indicators.
- Sufficient color contrast.
- Semantic HTML.
- Screen reader compatibility where applicable.

Accessibility improvements shall continue in future releases.

4.9 Browser Compatibility

Version 1 shall support the latest stable versions of:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox

Support for Safari and additional browsers may be introduced in later versions.

4.10 Responsive Design

The interface shall adapt to different screen sizes.

Desktop

Primary development experience.

Laptop

Fully supported.

Tablet

Panels shall collapse intelligently.

Essential development tools shall remain accessible.

Mobile

Limited functionality.

Supported activities include:

- Viewing projects
- Managing settings
- Basic workspace management
- Reviewing deployments

Code editing on mobile is intentionally limited in Version 1.

4.11 Security Requirements

Security shall be integrated into every layer of the application.

Authentication Security

The platform shall:

- Use JWT Authentication.
- Validate every protected request.
- Verify user permissions before granting access.
- Expire invalid sessions securely.

Password Security

Passwords shall:

- Never be stored in plain text.
- Be securely hashed.
- Never be returned through APIs.

Authorization

Every request shall verify:

- User identity.
- Organization membership.
- Workspace permissions.
- Project permissions.
- User role.

API Security

Backend APIs shall include:

- Input validation.
- Request sanitization.
- Protected endpoints.
- Consistent error handling.

General Security

The platform shall implement:

- CORS protection.
- Helmet security headers.
- XSS protection.

- CSRF protection where applicable.
- Rate limiting.
- Environment variable management.
- Secure file handling.

4.12 Data Integrity

The platform shall preserve user data throughout normal operations.

Project metadata, workspace information, and user settings shall remain consistent across sessions.

Unexpected interruptions shall not corrupt stored project information.

4.13 Logging

The backend shall maintain logs for:

- Authentication events.
- Server errors.
- Deployment operations.
- Critical system failures.
- Administrative actions.

Sensitive user information shall never be written to logs.

4.14 Error Handling

Errors shall:

- Provide clear messages to users.
- Avoid exposing internal implementation details.
- Be logged on the server.
- Guide users toward recovery when possible.

4.15 Code Quality Standards

All source code shall follow these principles:

- TypeScript for type safety.
- Strong typing throughout the application.
- Consistent formatting.
- Feature-based folder organization.
- Reusable components.
- Small, focused functions.
- Descriptive naming conventions.
- Minimal technical debt.

4.16 UI/UX Standards

Every interface within Nexus Cloud IDE shall follow a unified design language.

Visual Style

- Premium
- Professional
- Minimal
- Modern
- Consistent

Color Palette

Primary Background:

`#0F1115`

Secondary Background:

`#171A1F`

Surface:

`#20242B`

Primary Text:

`#FFFFFF`

Secondary Text:

`#9DA5B4`

Accent:

`#C58A42`

Hover Accent:

`#D69A4E`

Success:

`#4CAF50`

Error:

`#E65A5A`

Warning:

`#F2B94B`

Info:

`#4D8DFF`

Typography

Headings:

- Inter
- Bold

Body:

- Inter
- Regular

Editor:

- JetBrains Mono

Border Radius

Primary components shall use approximately **12px** rounded corners.

Shadows

Soft shadows shall create visual depth without overwhelming the interface.

Animations

Animations shall be subtle and purposeful.

Examples include:

- Page transitions
- Sidebar collapse
- Dialog appearance
- Hover effects
- Loading transitions

Animation duration shall generally remain between **200–300 ms**.

Interaction Philosophy

The interface shall prioritize:

- Clarity
- Consistency
- Predictability
- Responsiveness

Visual polish shall never compromise usability or performance.

4.17 Future Readiness

The architecture shall accommodate future capabilities, including:

- AI Agents
- Enterprise workspaces
- Advanced monitoring
- Third-party integrations
- Expanded deployment providers

- Mobile companion applications
- Enhanced collaboration features

Future enhancements should integrate with the existing architecture without requiring major structural changes.

Tab 5

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Nexus Cloud IDE

Version: 1.0
Document Status: Approved for Development

---

5. System Architecture & Technical Design

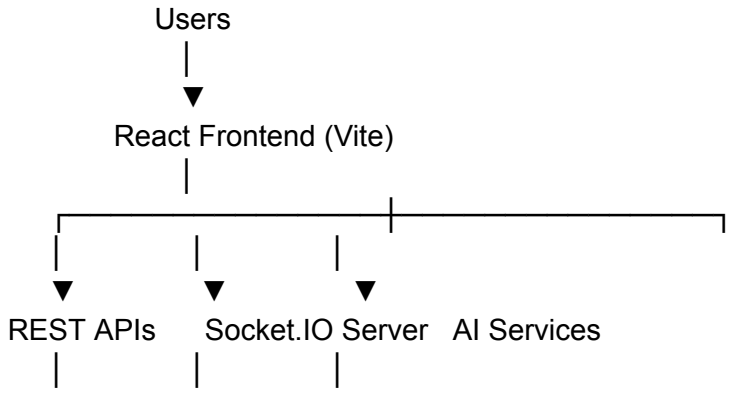
5.1 Architecture Overview

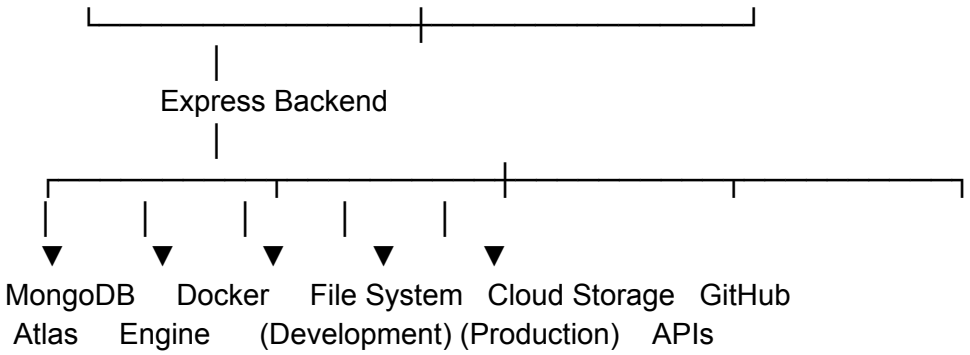
Nexus Cloud IDE shall follow a modern, scalable, modular, and production-ready architecture based on a client-server model.

The system is designed to separate responsibilities into independent layers, allowing each module to evolve without affecting the rest of the platform.

The architecture emphasizes maintainability, scalability, security, and performance while supporting future enterprise expansion.

5.2 High-Level System Architecture





5.3 Architectural Principles

The system shall follow the following principles:

- Modular Architecture
- Feature-Based Development
- Separation of Concerns
- Single Responsibility Principle
- Reusable Components
- Strong Typing
- Secure by Design
- API-First Development
- Future Scalability
- Clean Code Practices

5.4 Frontend Architecture

Technology Stack

The frontend shall use:

- React
- Vite
- TypeScript
- Tailwind CSS
- Redux Toolkit
- TanStack Query
- React Router
- Axios
- React Hook Form
- Zod

- Monaco Editor
- Framer Motion
- Lucide React

Frontend Responsibilities

The frontend shall handle:

- User Interface
- Navigation
- Authentication State
- API Communication
- Form Validation
- Theme Management
- Workspace Rendering
- Code Editor Integration
- State Management
- Real-Time Updates
- Client-side Routing

Frontend Folder Structure

```
frontend/
│
├── public/
│
├── src/
│   ├── app/
│   ├── assets/
│   ├── components/
│   ├── config/
│   ├── features/
│   ├── hooks/
│   ├── layouts/
│   ├── pages/
│   ├── providers/
│   ├── routes/
│   ├── services/
│   ├── store/
│   ├── styles/
│   ├── types/
│   ├── utils/
│   ├── lib/
```

```
│   ├── App.tsx
│   └── main.tsx
│
├── package.json
├── tsconfig.json
└── vite.config.ts
```

5.5 Backend Architecture

Technology Stack

The backend shall use:

- Node.js
- Express.js
- TypeScript
- MongoDB
- Mongoose
- JWT
- Bcrypt
- Socket.IO
- Docker
- Multer
- Cloudinary
- Swagger

Backend Responsibilities

The backend shall manage:

- Authentication
- Authorization
- User Management
- Organization Management
- Workspace Management
- Project Management
- Git Integration
- AI Requests
- Deployment
- Database Connections
- File Management
- Logging

- Security

Backend Folder Structure

```
backend/
│
├── src/
│   ├── config/
│   ├── controllers/
│   ├── middleware/
│   ├── models/
│   ├── routes/
│   ├── services/
│   ├── repositories/
│   ├── sockets/
│   ├── validators/
│   ├── utils/
│   ├── types/
│   ├── docs/
│   ├── app.ts
│   └── server.ts
│
├── package.json
└── tsconfig.json
```

5.6 Clean Architecture Layers

The backend shall follow a layered architecture.

Request

↓

Route

↓

Middleware

↓

Controller

↓

Service

↓

Repository

↓

Database

Responsibilities

Routes

Receive incoming requests and map them to controllers.

Middleware

Perform authentication, authorization, validation, logging, and error handling.

Controllers

Handle HTTP requests and responses.

Services

Contain business logic.

Repositories

Communicate with MongoDB.

Database

Store application data.

5.7 Database Design

MongoDB Atlas shall serve as the primary database.

MongoDB will store:

- Users
- Organizations
- Workspaces

- Projects
- Project Metadata
- AI Conversations
- Deployments
- Settings
- Roles
- Notifications
- Billing Information

Project source files will **not** be stored inside MongoDB.

5.8 File Storage Strategy

Development

During development, project files shall be stored on the local file system.

Advantages include:

- Faster development
- Easier debugging
- Simpler local execution

Production

Production deployments shall store project files in dedicated object storage.

Examples include:

- Cloudinary (for media assets)
- Amazon S3 (future)
- Compatible object storage providers

MongoDB shall only maintain references and metadata for stored files.

5.9 Authentication Flow

User

↓

Login Request

↓

Express API

↓

Validate Credentials

↓

Generate JWT

↓

Return Access Token

↓

Frontend Stores Token Securely

↓

Authenticated Requests

5.10 Authorization

Every protected request shall validate:

- User identity
- Organization membership
- Workspace access
- Project ownership
- Assigned role

Unauthorized access attempts shall be rejected.

5.11 API Design

The backend shall expose RESTful APIs.

Example endpoint groups:

/api/auth

/api/users

/api/organizations

/api/workspaces

/api/projects

/api/files

/api/editor

/api/git

/api/database

/api/deployments

/api/ai

/api/settings

API responses shall use consistent formats for success, validation errors, and server errors.

5.12 Real-Time Communication

Socket.IO shall provide real-time capabilities.

Version 1 will support:

- Shared terminal updates
- Collaboration events
- Workspace presence
- Toast event synchronization

Future versions may extend Socket.IO for:

- Live editing
- Live cursors

- Voice collaboration

5.13 State Management

Frontend state shall be divided into:

Redux Toolkit

Global application state.

Examples:

- Authentication
- User
- Theme
- Workspace
- Notifications

TanStack Query

Server state.

Examples:

- Projects
- Workspaces
- Deployments
- AI Requests
- Organization Data

Local Component State

Component-specific UI state.

Examples:

- Dialogs
- Menus
- Input fields
- Temporary selections

5.14 Deployment Architecture

Frontend

Hosted on:

- Vercel

Backend

Hosted on:

- Render

Database

Hosted on:

- MongoDB Atlas

Object Storage

Media assets:

- Cloudinary

Project files:

- Production object storage (future-ready)

Containerization

Docker shall be used for:

- Code execution
- Isolated runtime environments
- Deployment packaging

5.15 Security Architecture

The architecture shall implement:

- JWT Authentication
- Password Hashing (Bcrypt)
- Environment Variables
- CORS Protection
- Helmet
- Input Validation
- Rate Limiting
- XSS Protection
- Secure API Access
- Permission-Based Authorization

Security requirements shall apply uniformly across all backend services.

5.16 Scalability Strategy

The system architecture shall support:

- Horizontal backend scaling
- Additional deployment providers
- AI service expansion
- Enterprise organizations
- Future microservice extraction if required

The initial implementation shall remain a modular monolith to reduce operational complexity while preserving a clear migration path to microservices.

Tab 6

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Nexus Cloud IDE

Version: 1.0
Document Status: Approved for Development

---

6. Database Design & Data Models

6.1 Introduction

This section defines the logical database structure for Nexus Cloud IDE.

Version 1 uses **MongoDB Atlas** as the primary database management system. MongoDB has been selected due to its flexible document model, scalability, high performance, and compatibility with modern JavaScript/TypeScript applications.

The database design prioritizes maintainability, efficient querying, and future expansion.

6.2 Database Overview

MongoDB shall store application metadata rather than project source code.

Project source files are stored separately on the file system during development and object storage in production.

MongoDB shall maintain references, metadata, ownership information, permissions, and configuration.

6.3 Primary Collections

Version 1 shall include the following collections:

users

organizations

organizationMembers

workspaces

projects

projectFiles

deployments

gitRepositories

databaseConnections

aiConversations

notifications

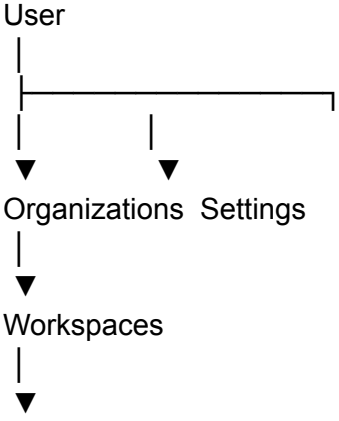
subscriptions

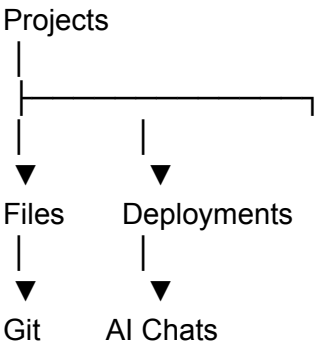
settings

activityLogs

Additional collections may be introduced without affecting the existing schema.

6.4 Entity Relationships





6.5 User Collection

Purpose

Stores user identity and authentication information.

Fields

Field	Type	Required
_id	ObjectId	Yes
fullName	String	Yes
username	String	Yes
email	String	Yes
passwordHash	String	Yes (Email Login)
avatar	String	No
authProvider	String	Yes
emailVerified	Boolean	Yes
createdAt	Date	Yes
updatedAt	Date	Yes

Relationships

One user may belong to:

- Multiple Organizations

- Multiple Workspaces
- Multiple Projects

6.6 Organization Collection

Purpose

Represents companies, teams, or personal organizations.

Fields

Field	Type
_id	ObjectId
name	String
logo	String
ownerId	ObjectId
createdAt	Date
updatedAt	Date

Each organization owns:

- Workspaces
- Members
- Projects
- Billing

6.7 Organization Members Collection

Purpose:

Stores role assignments.

Fields

- Organization ID

- User ID
- Role
- Joined Date

Supported Roles:

- Owner
- Admin
- Maintainer
- Developer
- Viewer

6.8 Workspace Collection

Purpose

Represents a complete development environment.

Fields

Field	Type
_id	ObjectId
organizationId	ObjectId
name	String
description	String
terminalEnabled	Boolean
aiEnabled	Boolean
createdBy	ObjectId
createdAt	Date

Relationships

Workspace contains:

- Projects
- AI Conversations

- Database Connections
- Deployments

6.9 Project Collection

Purpose

Represents software applications.

Fields

Field	Type
_id	ObjectId
workspaceId	ObjectId
name	String
template	String
language	String
visibility	String
gitEnabled	Boolean
deploymentEnabled	Boolean
createdBy	ObjectId
createdAt	Date

Supported Templates

- HTML
- CSS
- JavaScript
- React
- Next.js
- Node.js
- Express
- React + Express
- Python

- Java
- C++
- Empty

6.10 Project Files Collection

Although project files are stored outside MongoDB, metadata shall be maintained.

Stored Metadata

- File Name
- File Path
- Parent Folder
- Size
- Extension
- Last Modified
- Created Date
- Owner

Example

src/

components/

Navbar.tsx

pages/

Dashboard.tsx

6.11 AI Conversations Collection

Stores AI conversations for every workspace.

Fields

- Workspace ID
- User ID
- Conversation Title

- Messages
- Created Date
- Updated Date

Each workspace maintains its own independent conversation history.

6.12 Git Repository Collection

Stores Git metadata.

Fields include:

- Repository URL
- Current Branch
- Last Commit
- Provider
- Connected Account

6.13 Deployment Collection

Stores deployment history.

Fields include:

- Deployment Provider
- Status
- Live URL
- Build Logs
- Deployment Time
- User
- Project

Supported Providers

- Vercel
- Docker
- GitHub Pages

6.14 Database Connections Collection

Stores external database configurations.

Supported:

- MongoDB
- PostgreSQL
- SQLite

Stored Information

- Connection Name
- Provider
- Host
- Port
- Database Name
- Username

Passwords shall never be stored in plain text.

6.15 Notifications Collection

Stores notification history.

Version 1 includes only toast notifications.

Examples

- Login Successful
- Project Saved
- Deployment Complete

Future versions may extend this into a complete notification center.

6.16 Subscription Collection

Stores billing information.

Plans

- Free

- Pro

Tracks

- Plan
- Renewal Date
- Storage Usage
- Project Limits

6.17 Settings Collection

Stores user preferences.

Examples

- Theme
- Editor Preferences
- Font Size
- Auto Save
- Default Workspace

6.18 Activity Logs

Stores important actions.

Examples

- Login
- Logout
- Workspace Creation
- Project Creation
- Deployment
- Git Push

Activity logs improve debugging and auditing.

6.19 Indexing Strategy

Indexes shall be created for frequently queried fields.

Examples include:

- Email
- Username
- Organization ID
- Workspace ID
- Project ID
- User ID
- Created Date

Proper indexing shall improve query performance as the platform scales.

6.20 Validation Rules

All database operations shall enforce validation before persistence.

Examples include:

- Required fields
- Unique email addresses
- Valid role assignments
- Valid workspace references
- Valid organization ownership

Invalid data shall never be written to the database.

6.21 Soft Delete Strategy

Instead of permanently deleting critical records immediately, the platform shall support soft deletion where appropriate.

Soft-deleted resources may include:

- Projects
- Workspaces
- Organizations

This allows recovery and prevents accidental data loss.

6.22 Data Lifecycle

Every entity follows a predictable lifecycle.

Example for a project:

Created

Active

Archived

Soft Deleted

Permanently Deleted (Administrative Cleanup)

6.23 Future Database Expansion

The schema has been designed to accommodate future additions without major restructuring.

Planned future collections may include:

- AI Agents
- Team Permissions
- Monitoring Metrics
- Plugin Registry
- Marketplace
- Organization Invitations
- Audit Trails
- Cloud Resources

Tab 7

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Nexus Cloud IDE

Version: 1.0
Document Status: Approved for Development

---

7. UI/UX Design Specification

7.1 Introduction

The user interface of Nexus Cloud IDE is a defining aspect of the platform. It is intended to deliver a premium, modern, and professional experience that encourages users to remain productive within the platform for extended periods.

The design philosophy emphasizes simplicity, consistency, responsiveness, and elegance. Every interface element must contribute to a cohesive user experience while avoiding unnecessary visual complexity.

The interface shall be inspired by the usability standards of leading cloud development environments while maintaining its own distinct identity.

7.2 Design Philosophy

The interface shall follow these guiding principles:

- Professional before decorative.
- Function before ornamentation.
- Consistent layouts across all modules.
- Minimal visual noise.
- Smooth interactions.
- Clear visual hierarchy.
- Immediate user feedback.
- Predictable navigation.

- Accessibility by design.
- Responsive behavior across supported devices.

The objective is to create an interface that users perceive as polished, trustworthy, and efficient.

7.3 Visual Identity

The visual identity of Nexus Cloud IDE shall communicate professionalism and technical precision.

The application shall adopt a dark theme as the default and only theme in Version 1, with carefully balanced contrast to reduce eye strain during extended coding sessions.

The overall appearance should be modern, clean, and understated rather than visually extravagant.

7.4 Color System

Primary Background

`#0F1115`

Used for the main application background.

Secondary Background

`#171A1F`

Used for sidebars, panels, and navigation areas.

Surface Background

`#20242B`

Used for cards, dialogs, dropdowns, and floating panels.

Border Color

`rgba(255,255,255,0.08)`

Used consistently across components to create subtle separation.

Primary Text

`#FFFFFF`

Used for headings and primary content.

Secondary Text

`#9DA5B4`

Used for descriptions, metadata, and secondary information.

Accent Color

`#C58A42`

Used for:

- Active navigation
- Primary buttons
- Focus indicators
- Selected elements

Hover Accent

`#D69A4E`

Used for hover interactions and highlighted actions.

Success

`#4CAF50`

Used for:

- Successful deployments
- Saved changes
- Git success messages
- Confirmation indicators

Error

`#E65A5A`

Used for:

- Validation errors
- Failed deployments
- Failed authentication
- Critical alerts

Warning

`#F2B94B`

Used for:

- Unsaved changes
- Confirmation dialogs
- Warning banners

Information

`#4D8DFF`

Used for:

- Informational messages
- Tooltips
- Helpful guidance

7.5 Typography

The typography system shall prioritize readability and consistency.

Primary Font

Inter

Used for:

- Navigation
- Cards
- Buttons
- Dialogs
- Forms
- Documentation

Heading Weight

700

Body Weight

400

Button Weight

500

Code Font

JetBrains Mono

Used exclusively within:

- Monaco Editor
- Terminal
- Code blocks
- Console output

7.6 Spacing System

The application shall follow a consistent spacing scale.

Preferred spacing values include:

- 4px
- 8px
- 12px
- 16px
- 20px
- 24px
- 32px

All layouts should align to this spacing system to ensure visual consistency.

7.7 Border Radius

The platform shall use rounded corners consistently.

Standard radius:

12px

Exceptions may include:

- Small badges (8px)
- Circular avatars (50%)
- Buttons (10–12px)

7.8 Shadows

Shadows shall be soft and subtle.

They should provide depth without creating excessive contrast.

Heavy shadows, glowing effects, or dramatic elevation changes shall not be used.

7.9 Glassmorphism

Glassmorphism shall be applied sparingly.

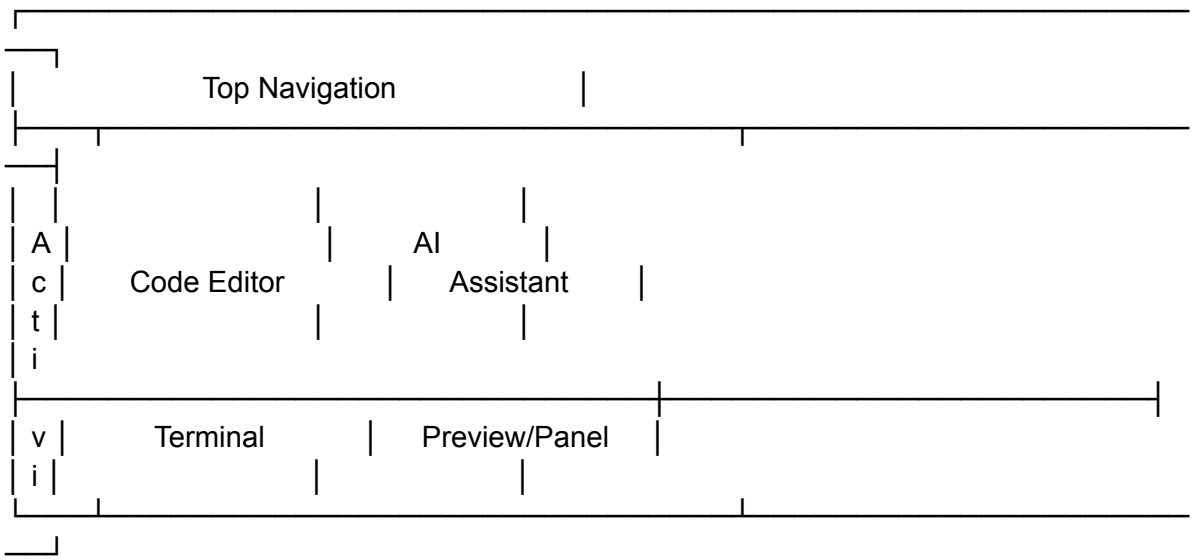
Approved use cases include:

- Floating command palettes
- Modal dialogs
- Context menus
- Workspace switchers

The primary interface should remain solid and highly readable.

7.10 Layout Structure

The primary application layout consists of five major regions.



Every primary module should remain accessible without requiring excessive navigation.

7.11 Top Navigation Bar

The top navigation bar shall remain fixed while users navigate within the application.

It shall include:

- Nexus Cloud IDE logo
- Active workspace selector
- Global search (future)
- Project name
- Git status
- Notifications
- Share button
- Run button
- Deploy button
- User profile
- Settings shortcut

The navigation bar shall maintain a compact height to maximize editor space.

7.12 Activity Bar

The Activity Bar shall remain fixed on the left side of the interface.

It provides quick access to major modules.

Version 1 includes:

- Explorer
- Search (future)
- Git
- Database Explorer
- AI Assistant
- Terminal
- Deployments
- Settings

Each icon shall include:

- Hover tooltip
- Active indicator
- Keyboard accessibility

Icons shall use the Lucide icon library unless a custom icon is required.

7.13 Sidebar

The sidebar shall display contextual information depending on the active module.

Examples:

Explorer → Project files

Git → Source control

Database → Connections

AI → Conversations

Settings → Configuration options

Only one sidebar panel shall remain active at a time.

7.14 Code Editor

The Monaco Editor shall occupy the majority of the available workspace.

The editor shall support:

- Multiple tabs
- Split view
- Breadcrumb navigation
- Minimap
- Line numbers
- Auto-save
- Unsaved indicators
- Syntax highlighting
- Bracket matching
- Auto indentation

The editor shall remain the visual focus of the interface.

7.15 Bottom Panel

The bottom panel provides access to development tools.

Version 1 includes:

- Terminal
- Output
- Problems
- Build Logs

The panel shall be:

- Resizable
- Collapsible
- Docked to the bottom

7.16 Right Panel

The right panel contains the AI Assistant.

It shall support:

- Conversation history
- Markdown rendering
- Code snippets
- Suggested improvements
- Documentation assistance

The panel may be resized or collapsed by the user.

7.17 Dialogs

Dialogs shall be used for:

- Creating projects
- Deleting resources
- Workspace settings
- User invitations
- Deployment configuration

Dialogs shall include:

- Clear title
- Short description
- Primary action
- Secondary action
- Keyboard shortcuts where appropriate

7.18 Buttons

Buttons shall be categorized into:

- Primary
- Secondary
- Destructive
- Ghost
- Icon-only

Each category shall maintain a consistent visual style throughout the platform.

7.19 Forms

Forms shall provide:

- Real-time validation
- Clear labels
- Helpful error messages
- Required field indicators
- Keyboard-friendly navigation

Validation should occur without disrupting user workflow.

7.20 Navigation Philosophy

Users should be able to reach any major feature within three interactions or fewer.

Navigation shall remain predictable and consistent across all modules.

No feature should be hidden behind unnecessary nesting or complex menus.

7.21 Animation Standards

Animations shall enhance usability without distracting the user.

Standard animation duration:

200–300 milliseconds

Supported animation types:

- Fade
- Slide
- Scale
- Panel collapse
- Dialog appearance
- Hover transitions

Animations shall never interfere with productivity.

7.22 Responsive Design

Desktop

Provides the complete development experience.

Laptop

Maintains full functionality with adaptive spacing.

Tablet

Side panels collapse into overlays while preserving editor usability.

Mobile

Supports:

- Dashboard
- Projects
- Settings
- Workspace management

The coding experience is intentionally limited on mobile devices.

7.23 Accessibility Standards

The interface shall support:

- Keyboard navigation
- Visible focus states
- Sufficient color contrast
- Semantic HTML
- Screen reader compatibility where applicable

Accessibility improvements shall continue in future releases.

7.24 UI Consistency Rules

To maintain a premium experience, the following rules apply across the application:

- No inconsistent spacing.
- No mixed border radius values without purpose.
- No inconsistent typography.
- No unnecessary gradients.
- No neon or overly saturated colors.
- No decorative animations that reduce usability.
- Every component must follow the established design system.

Tab 8

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Nexus Cloud IDE

Version: 1.0
Document Status: Approved for Development

---

8. Development Standards & Implementation Guidelines

8.1 Introduction

This section establishes the engineering standards, development practices, and implementation guidelines that shall govern the development of Nexus Cloud IDE.

The objective is to ensure that the platform remains maintainable, scalable, secure, and consistent throughout its development lifecycle.

Every contributor to the project shall follow the standards defined in this section.

8.2 Development Philosophy

Development shall prioritize:

- Clean architecture over shortcuts.
- Readable code over clever code.
- Reusable components over duplication.
- Scalability over temporary solutions.
- Maintainability over rapid implementation.
- Consistency across the entire codebase.

Every feature added to the platform must align with the product vision and architectural principles established in this SRS.

8.3 Coding Standards

The codebase shall adhere to the following principles:

- Use TypeScript for all frontend and backend code.
- Prefer descriptive names for variables, functions, and classes.
- Keep functions focused on a single responsibility.
- Avoid deeply nested logic.
- Eliminate unnecessary code duplication.
- Write self-explanatory code whenever possible.
- Add comments only where business logic requires clarification.

8.4 Folder Organization

The project shall use a feature-based architecture to improve modularity and maintainability.

Each feature shall contain its own:

- Components
- Hooks
- Services
- Types
- Utilities
- API integrations (where applicable)

Shared functionality shall reside in common directories.

8.5 Naming Conventions

Files

React components:

ProjectCard.tsx
WorkspaceSidebar.tsx
DeploymentDialog.tsx

Hooks:

useWorkspace.ts
useProjects.ts
useEditor.ts

Services:

auth.service.ts
project.service.ts
deployment.service.ts

Types:

user.types.ts
workspace.types.ts
project.types.ts

Utilities:

formatDate.ts
generateSlug.ts
storage.ts

Variables

Use camelCase.

Example:

currentWorkspace
activeProject
selectedFile

Components

Use PascalCase.

Example:

DashboardLayout
WorkspaceNavbar
EditorTabs

Constants

Use UPPER_SNAKE_CASE.

Example:

MAX_PROJECT_LIMIT
DEFAULT_THEME
API_BASE_URL

8.6 API Design Standards

REST endpoints shall follow predictable naming conventions.

Examples:

GET /api/projects
POST /api/projects
GET /api/projects/:id
PATCH /api/projects/:id
DELETE /api/projects/:id

Endpoints shall use nouns rather than verbs whenever practical.

API Response Format

Successful responses:

```
{
  "success": true,
  "message": "Project created successfully.",
  "data": {}
}
```

Error responses:

```
{
  "success": false,
  "message": "Project not found.",
  "errors": []
}
```

Validation errors shall provide sufficient information for frontend handling while avoiding exposure of internal implementation details.

8.7 Error Handling Standards

Errors shall be categorized into:

- Validation Errors
- Authentication Errors
- Authorization Errors
- Business Logic Errors
- Database Errors
- External Service Errors
- Internal Server Errors

Each category shall return appropriate HTTP status codes and user-friendly messages.

8.8 Git Workflow

The project shall use Git as the version control system.

Branches

- `main` – Production-ready code
- `develop` – Integration branch
- `feature/<feature-name>` – New features
- `bugfix/<bug-name>` – Bug fixes
- `hotfix/<issue-name>` – Production fixes

Commit Messages

Commit messages should be clear and concise.

Examples:

feat: add workspace creation

fix: resolve sidebar collapse issue

refactor: optimize authentication flow

docs: update API documentation

style: improve dashboard spacing

test: add project service tests

8.9 Code Review Standards

Every major feature should be reviewed before integration.

Reviews should verify:

- Correctness
- Readability
- Performance
- Security
- Consistency with architecture
- Test coverage (where applicable)

Constructive feedback should focus on improving code quality and maintainability.

8.10 Dependency Management

Only dependencies that provide clear value shall be added to the project.

Before introducing a new dependency, developers should consider:

- Community adoption
- Maintenance status
- Security
- Bundle size
- Compatibility
- Long-term support

Unnecessary dependencies should be avoided.

8.11 Environment Configuration

Sensitive configuration values shall be stored using environment variables.

Examples include:

- Database connection strings
- JWT secrets
- OAuth credentials
- API keys
- Cloud storage credentials

Environment files shall never be committed to version control.

8.12 Logging Standards

Server-side logs shall include:

- Authentication events
- Deployment operations
- API failures
- Database errors
- Critical application events

Logs shall exclude:

- Plain text passwords
- Access tokens
- Secret keys
- Sensitive personal information

8.13 Documentation Standards

The project shall maintain comprehensive documentation.

Documentation should include:

- Project setup
- Folder structure
- API reference
- Deployment guide
- Environment configuration
- Coding standards
- Architecture overview
- Feature documentation

Documentation shall remain synchronized with implementation.

8.14 Testing Guidelines

Version 1 shall prioritize:

- Unit testing for core business logic
- API testing for backend endpoints
- Manual UI testing for critical workflows

Future versions may expand automated testing to include:

- Integration testing
- End-to-end testing
- Performance testing
- Accessibility testing

8.15 Build & Deployment Guidelines

Before deployment, the application shall satisfy the following criteria:

- Successful production build
- No TypeScript compilation errors
- No critical linting issues
- Environment variables configured
- Security checks completed
- Database migrations applied (if required)

Deployments should follow a repeatable and documented process.

8.16 Performance Optimization Standards

Developers should optimize performance through:

- Code splitting
- Lazy loading
- Memoization where beneficial
- Efficient state management
- Image optimization
- API request caching
- Virtualized rendering for large datasets

Performance optimizations should not compromise code readability.

8.17 Security Best Practices

Security shall be integrated throughout development.

Key practices include:

- Validate all user input
- Sanitize incoming data
- Enforce authentication on protected routes
- Verify permissions before sensitive actions
- Store secrets securely
- Keep dependencies updated
- Monitor known vulnerabilities

8.18 Release Management

Each release shall include:

- Version number
- Release notes
- Feature summary
- Bug fixes
- Known issues
- Upgrade instructions (if applicable)

Stable releases shall be tagged in version control.

8.19 Continuous Improvement

The project shall evolve through iterative improvements while preserving backward compatibility where practical.

Architectural changes should be carefully evaluated to ensure they align with the long-term vision of Nexus Cloud IDE.

8.20 Acceptance Criteria for Version 1

Version 1 of Nexus Cloud IDE shall be considered complete when:

- Users can register and authenticate securely.
- Organizations and workspaces can be created and managed.
- Projects can be created from supported templates.
- The Monaco Editor provides a stable coding experience.
- The AI Assistant delivers project-aware code suggestions.
- Git integration supports core workflows.
- Database Explorer allows viewing and querying supported databases.
- One-click deployment works with supported providers.
- Collaboration features operate as specified.
- The application adheres to the defined UI/UX standards.
- The platform demonstrates production-ready architecture and code quality.

Tab 9

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Nexus Cloud IDE

Version: 1.0
Document Status: Final

---

9. Testing Strategy & Quality Assurance

9.1 Purpose

The purpose of testing is to ensure that Nexus Cloud IDE is stable, secure, reliable, performant, and ready for production use.

Testing shall be performed continuously throughout development rather than only before release.

9.2 Testing Objectives

The testing process shall verify:

- Functional correctness
- User interface consistency
- API reliability
- Security
- Performance
- Cross-browser compatibility
- Data integrity
- Deployment reliability

9.3 Testing Levels

Unit Testing

Unit tests shall validate individual functions, utilities, hooks, and services.

Examples:

- Authentication utilities
- Permission checks
- Validation logic
- Helper functions

Integration Testing

Integration tests shall verify communication between:

- Frontend and Backend
- Backend and Database
- Backend and External Services

End-to-End Testing

Complete user workflows shall be validated.

Examples include:

- User Registration
- Login
- Workspace Creation
- Project Creation
- Git Push
- Deployment
- AI Interaction

Manual Testing

Manual testing shall verify:

- Visual consistency
- Responsive layouts
- User experience
- Keyboard navigation
- Accessibility

9.4 Browser Testing

Version 1 shall be tested on:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox

9.5 Performance Testing

Performance evaluation shall include:

- Initial page load
- Workspace loading
- Large project handling
- API response time
- Editor responsiveness

9.6 Security Testing

Security validation shall include:

- Authentication
- Authorization
- JWT validation
- Input sanitization
- API protection
- Rate limiting

9.7 Acceptance Testing

A feature shall be considered complete only if:

- Functional requirements are satisfied.
- UI matches the design system.
- No critical defects remain.
- Performance meets defined standards.
- Documentation has been updated.

---

10. Deployment & Infrastructure

10.1 Development Environment

Development shall use:

- Windows / Linux / macOS
- Visual Studio Code
- Node.js
- Docker Desktop
- Git
- MongoDB Atlas

10.2 Production Infrastructure

Frontend

- Vercel

Backend

- Render

Database

- MongoDB Atlas

Media Storage

- Cloudinary

Container Runtime

- Docker

10.3 Environment Variables

Configuration shall include:

- Database URI

- JWT Secret
- GitHub OAuth Credentials
- Google OAuth Credentials
- Cloudinary Credentials
- AI API Keys
- Deployment Provider Keys

No secrets shall be committed to version control.

10.4 CI/CD

Future releases should automate:

- Build
- Testing
- Linting
- Deployment
- Release tagging

10.5 Backup Strategy

Regular backups shall be maintained for:

- Database
- User settings
- Project metadata
- Billing information

Project source files shall follow the backup policies of the configured storage provider.

---

11. Project Roadmap & Development Phases

Development shall proceed incrementally through well-defined phases.

Phase 1 – Foundation

- Environment Setup
- Project Initialization
- Folder Structure
- Design System
- Routing
- State Management
- Authentication Foundation

Phase 2 – Core Platform

- Dashboard
- Organizations
- Workspaces
- Project Management
- File Explorer
- Monaco Editor
- Terminal

Phase 3 – Development Features

- Git Integration
- Database Explorer
- AI Assistant
- Templates
- Auto Save
- Editor Improvements

Phase 4 – Cloud Features

- Deployment
- Collaboration
- Shared Terminal
- Shared Preview
- Workspace Invitations

Phase 5 – Platform Features

- Billing

- Admin Dashboard
- Settings
- Notifications
- Theme System

Phase 6 – Production Readiness

- Optimization
- Testing
- Documentation
- Security Review
- Deployment
- Final Release

---

12. Risks, Assumptions & Future Scope

Risks

Potential project risks include:

- Third-party API changes
- Cloud service outages
- Browser compatibility issues
- Large project performance
- AI service limitations
- Infrastructure costs

Mitigation strategies shall be planned for each identified risk.

Assumptions

The project assumes:

- Stable internet connectivity
- Modern browser support
- Docker availability
- Supported cloud providers remain operational
- Users possess basic programming knowledge

Future Scope

Future releases may include:

- AI Agents
- Enterprise Organizations
- Live Collaborative Editing
- Voice Collaboration
- Plugin Marketplace
- Mobile Companion Application
- Monitoring Dashboard
- Team Analytics
- Multi-Workspace Organizations
- Cloud Resource Management
- Kubernetes Deployment
- Multi-Cloud Deployment
- AI Code Review
- AI Architecture Generation
- AI Project Planning

These enhancements are intentionally outside the scope of Version 1.

---

13. Appendices & Glossary

Glossary

Organization
A logical container representing an individual, team, startup, or company.

Workspace
A complete development environment that groups projects, collaboration, AI conversations, terminal sessions, database connections, and deployment settings.

Project
A software application created and managed within a workspace.

Deployment
The process of publishing an application to a hosting platform.

AI Assistant
An intelligent, workspace-aware assistant that understands the project context and provides development suggestions.

Template
A predefined project structure used to accelerate project creation.

Terminal
A command-line interface integrated into the workspace.

Database Explorer
A visual interface for connecting to and querying supported databases.

Acronyms

Acronym	Meaning
IDE	Integrated Development Environment
API	Application Programming Interface
UI	User Interface
UX	User Experience
JWT	JSON Web Token
REST	Representational State Transfer
CI/CD	Continuous Integration / Continuous Deployment
CRUD	Create, Read, Update, Delete
SaaS	Software as a Service
ODM	Object Document Mapper

References

The development of Nexus Cloud IDE is inspired by the usability and workflows of modern development platforms, while maintaining its own architecture and identity.

Reference inspirations include:

- Visual Studio Code
- GitHub Codespaces
- Replit
- Cursor
- StackBlitz
- CodeSandbox
- Linear

- Vercel Dashboard

These references inform usability and workflow concepts only. Nexus Cloud IDE defines its own product vision, feature integration, architecture, and user experience.

Document Approval

Project Name: Nexus Cloud IDE

Document: Software Requirements Specification (SRS)

Version: 1.0

Status: Approved for Development

Prepared By: Harshini (Project Owner) & ChatGPT (Technical Design Assistant)

Document Freeze: Version 1.0

🎉 SRS Version 1.0 Complete

This SRS is now the **official contract** for the project. From this point onward:

- No implementation should contradict the SRS unless the SRS is formally updated.
- Every feature, screen, API, database model, and UI decision should trace back to this document.
- New ideas should be evaluated against the project's core vision before being added.

Given everything we've discussed over the past sessions, I would also recommend maintaining three living documents alongside the SRS:

1. **Software Requirements Specification (SRS)** – the "what" and "why" (completed).
2. **Software Design Document (SDD)** – the detailed implementation blueprint (UI flows, API contracts, database schemas, sequence diagrams, architecture diagrams, etc.).
3. **Development Roadmap & Task Tracker** – a milestone-based checklist that breaks the entire project into hundreds of manageable tasks so we can build Nexus Cloud IDE systematically without confusion.

With these three documents, you'll have a professional documentation set similar to what is used for real SaaS products before development begins.

Tab 10

Tab 11

Tab 12

Tab 13